QDQ - static quantization

Monday, May 11, 2026
6:18 PM

YOLOX FP32 → ONNX ⟶ INT8 Quantization

▼ 1.Folder Structure

```
Quant/
__venv/
-yolox/
__model/
| |- best_ckpt.pth
| |_best_ckpt.onnx
| +-Mondeleze_int8.onnx
| +-yolox_voc_s3.py
| H-classes.txt
| |— infer_fp32.py
| |--quantize_int8.py
| Ł eval_onnx_map.py
⊢- calibration_images/
. datasets/
└ VOCdevkit/
L-VOC2012/
†—JPEGlmages/
-Annotations/
← ImageSets/
-Main/
← valid.txt
```

▼ 2. Environment Setup

Python Environment

Python 3.10 virtual environment was used.

First clone the official YOLOX repo and setup the environment

```
Installed Packages
pip install torch torchvision
```

```
pip install onnx onnxruntime-gpu onnxsim
pip install opencv-python numpy tqdm lxml loguru psutil
```

▼ 3. Exporting YOLOX .pth → ONNX

Input Files

Experiment File
model/yolox_voc_s3.py

Checkpoint
model/best_ckpt.pth

Export Command

```
Executed from inside YOLOX repository:
cd YOLOX
python tools/export_onnx.py }
-f ../model/yolox_voc_s3.py }
-c ../model/best_ckpt.pth }
–output-name ../model/best_ckpt.onnx }
–opset 18
```

Export Technique Used

FP32 Static Export
The PyTorch YOLOX model was exported into a static FP32 ONNX graph.
ONNX Opset

Opset = 18

Opset 18 was selected for better ONNX Runtime compatibility.

▼ 4. FP32 ONNX Inference Validation

Goal

Verify:

- ONNX model loads correctly
- Input preprocessing works
- Detection outputs are valid
- Bounding boxes render correctly

Inference Runtime

Runtime Used - ONNX Runtime

Execution Provider - providers=[“CPUExecutionProvider”]

GPU inference initially failed because CUDA libraries were missing.

Preprocessing Technique

YOLOX letterbox preprocessing was used:

- Resize while preserving aspect ratio
- Pad with value 114
- Convert HWC → CHW
- Float32 tensor
- Shape:
  $1 \times 3 \times 640 \times 640$

Postprocessing Technique

The following were used:
Confidence Threshold
CONF_THRES $= 0.25$

NMS Threshold
NMS_THRES $= 0.45$

Non-Maximum Suppression
Class-aware NMS was applied.

Result

Bounding boxes were successfully visualized.
The exported ONNX model behaved correctly.

▼ 5. VOC Dataset Setup

Dataset structure used:
datasets/VOCdevkit/VOC2012/

Required Components
Images
JPEGImages/

VOC XML Annotations
Annotations/

Validation List
ImageSets/Main/valid.txt

Classes
8 classes were used:
person
dock_open
truck
dock_closed
forklift
box_in_hand
pallet_load
no_truck

▼ 6. FP32 ONNX Evaluation

Measure:

- mAP@50
- mAP@70
- Inference latency

Evaluation Settings
Confidence Threshold

CONF_THRES = 0.001

Low threshold used intentionally for proper AP curve generation.
NMS Threshold
NMS_THRES = 0.65

IoU Thresholds
IOU_THRESHOLDS = [0.5, 0.7]

▼ 7. Understanding mAP Metrics

Intersection over Union (IoU)
IoU formula:
[
$IoU = \frac{\text{Area (Detection} \cap \text{GroundTruth)}}{\text{Area (Detection UGroundTruth)}}$]
Precision
[
Precision = $\frac{TP}{TP+FP}$]
Recall
[
Recall = $\frac{TP}{TP+FN}$]
Average Precision
[
$AP = \int_0^1$ Precision (Recall), dRecall]
Mean Average Precision
[
$mAP = \frac{1}{N \sum_{\{i=1\}}^{\{N\}} AP}$]
Where:

- N = number of classes
- AP_i = AP of class i

▼ 8. FP32 ONNX Evaluation Results

Per-Class AP @50
Class	AP
:—	:—:
person	0.9086

dock_open	0.9091
truck	0.9091
dock_closed	1.0000
forklift	0.9090
box_in_hand	0.7462
pallet_load	0.9086
no_truck	0.9974

FP32 Final Metrics

Metric	Value
mAP@50	0.9110
mAP@70	0.8713
Inference	25.79 ms/img
Model Size	35.8 MB

▼ 9. INT8 Quantization

Goal

Convert FP32 ONNX into INT8 ONNX.
Objectives:

- Reduce model size
- Reduce memory bandwidth
- Preserve accuracy
- Improve deployment efficiency

▼ 10. Quantization Technique Used

Quantization Type

Static Post Training Quantization (PTQ)
The model was quantized after training.
No retraining or QAT was used.

Quantization Format

QDQ Format
quant_format = QuantFormat.QDQ

QDQ inserts:

- QuantizeLinear
- DequantizeLinear
  operators around tensors.

Activation Type

activation_type = QuantType.QInt8

Weight Type

weight_type = QuantType.QInt8

Calibration Method

Entropy Calibration
calibrate_method = CalibrationMethod.MinMax
Tried using Entropy first but as PC is getting stuck we changed to MinMax
Entropy calibration:

- produces better accuracy
- analyzes activation distributions
- more computationally expensive

Compared to MinMax calibration:

Method	Accuracy	Speed
MinMax	Lower	Faster
Entropy	Higher	Slower

Per-Channel Quantization

per_channel = True

Per-channel quantization:

- quantizes each weight channel separately
- preserves accuracy better
- increases quantization complexity

Calibration Dataset

Calibration images were stored in:
Quant/calibration_images/

400 calibration images were used.

▼ 11. Quantization Command

python quantize_int8.py

The script:

1. Loaded FP32 ONNX
2. Read calibration images
3. Collected activation statistics
4. Generated INT8 QDQ graph
5. Saved quantized model

▼ 12. INT8 Inference Validation

Runtime

ONNX Runtime CPU

The quantized model successfully:

- loaded
- executed
- produced detections
- preserved localization quality

▼ 13. INT8 Evaluation Results

Per-Class AP @50

Class	AP
person	0.9084
dock_open	0.9091

truck	0.9091
dock_closed	1.0000
forklift	0.9090
box_in_hand	0.7282
pallet_load	0.9086
no_truck	0.9974

INT8 Final Metrics

Metric	Value
mAP@50	0.9087
mAP@70	0.8607
Inference	69.62 ms/img
Model Size	9.3 MB

▼ 14. FP32 vs INT8 Comparison

Metric	FP32 ONNX	INT8 ONNX
Model Size	35.8 MB	9.3 MB
mAP@50	0.9110	0.9087
mAP@70	0.8713	0.8607
Inference	25.79 ms/img	69.62 ms/img
Runtime	CPU FP32	CPU QDQ INT8

▼ 15. Accuracy Analysis

Accuracy Drop
mAP @ 50
[
$0.9110 - 0.9087 = 0.0023$]
Very small accuracy loss.
mAP @ 70
[
$0.8713 - 0.8607 = 0.0106$]
Slight localization degradation at stricter IoU.

▼ 16. Why INT8 Became Slower

INT8 model was slower because:

1. CPU Inference

Inference was performed on CPU.
CUDAExecutionProvider failed due to missing CUDA libraries.
Therefore:
CPU FP32 vs CPU QDQ INT8
was benchmarked.

2. QDQ Overhead

QDQ format inserts:

- QuantizeLinear
- DequantizeLinear
  operations throughout the graph.
  This adds overhead on CPU.

3. Per-Channel Quantization

Per-channel quantization improved accuracy but increased compute overhead.

4. ONNX Runtime CPU Optimizations

ONNX Runtime CPU FP32 kernels are highly optimized using:

- AVX2

- AVX512
- SIMD vectorization

Therefore FP32 CPU inference can outperform QDQ INT8 CPU inference.

▼ 17. Expected Real INT8 Acceleration

True INT8 acceleration typically requires:

TensorRT INT8

Pipeline:
ONNX → TensorRT Engine → INT8 Tensor Cores

Expected improvements:

- lower latency
- lower memory bandwidth
- GPU tensor core acceleration

Final Metrics Summary

Metric	FP32	INT8
Size	35.8 MB	9.3 MB
mAP@50	0.9110	0.9087
mAP@70	0.8713	0.8607
Inference	25.79 ms/img	69.62 ms/img
Quantization	None	Static PTQ QDQ
Calibration	None	Minmax
Weight Quant	FP32	Per-channel INT8
Runtime	ONNX Runtime CPU	ONNX Runtime CPU